

Análisis DevOps

2026-08-28

```
set.seed(123)
n_filas <- 5000

build_time_min <- round(runif(n_filas, min = 1.5, max = 45.0), 2)
deploy_time_min <- round(runif(n_filas, min = 0.5, max = 20.0), 2)
commit_size_loc <- rpois(n_filas, lambda = 150)
num_bugs <- rpois(n_filas, lambda = 1.2)
test_coverage_pct <- round(runif(n_filas, min = 40.0, max = 100.0), 2)
ticket_resolution_h <- round(rexp(n_filas, rate = 1/24), 2)

team <- sample(c("Alpha", "Beta", "Gamma", "Delta"), n_filas, replace = TRUE)
module <- sample(c("auth", "api", "ui", "database", "core"), n_filas, replace = TRUE)

niveles_prioridad <- c("baja", "media", "alta", "critica")
priority <- factor(
  sample(niveles_prioridad, n_filas, replace = TRUE, prob = c(0.4, 0.35, 0.15, 0.1)),
  levels = niveles_prioridad,
  ordered = TRUE
)

deploy_status <- sample(
  c("success", "failed", "rolled_back"),
  n_filas,
  replace = TRUE,
  prob = c(0.85, 0.10, 0.05)
)

devops_metrics <- data.frame(
  build_time_min,
  deploy_time_min,
  commit_size_loc,
  num_bugs,
  test_coverage_pct,
  ticket_resolution_h,
  team,
  module,
  priority,
  deploy_status
)

write.csv(devops_metrics, "devops_metrics.csv", row.names = FALSE)
```

Fase 1: Importación e inspección

En esta primera etapa importamos el conjunto de datos de métricas DevOps para inspeccionar su estructura general y detectar la cantidad de valores nulos presentes en cada columna.

```
library(dplyr)
library(readr)
library(tidyr)

datos_brutos <- read_csv("devops_metrics.csv")

## Rows: 5000 Columns: 10
## -- Column specification -----
## Delimiter: ","
## chr (4): team, module, priority, deploy_status
## dbl (6): build_time_min, deploy_time_min, commit_size_loc, num_bugs, test_co...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
glimpse(datos_brutos)
```

```
## Rows: 5,000
## Columns: 10
## $ build_time_min      <dbl> 14.01, 35.79, 19.29, 39.91, 42.41, 3.48, 24.47, 40~
## $ deploy_time_min    <dbl> 5.35, 19.79, 14.48, 13.21, 5.19, 2.19, 6.29, 11.07~
## $ commit_size_loc    <dbl> 143, 144, 168, 161, 154, 157, 152, 144, 163, 146, ~
## $ num_bugs           <dbl> 0, 2, 0, 0, 0, 0, 3, 3, 1, 3, 1, 2, 2, 0, 1, 2, 1,~
## $ test_coverage_pct  <dbl> 66.69, 54.81, 59.34, 53.03, 50.89, 62.63, 90.94, 9~
## $ ticket_resolution_h <dbl> 29.17, 6.18, 19.48, 21.42, 149.31, 15.38, 25.20, 4~
## $ team               <chr> "Beta", "Beta", "Alpha", "Delta", "Delta", "Gamma"~
## $ module             <chr> "api", "database", "api", "core", "database", "aut~
## $ priority           <chr> "critica", "media", "baja", "media", "media", "baj~
## $ deploy_status      <chr> "success", "success", "failed", "failed", "success~
```

```
summary(datos_brutos)
```

```
## build_time_min  deploy_time_min  commit_size_loc    num_bugs
## Min.   : 1.50    Min.   : 0.500    Min.   :110.0    Min.   :0.000
## 1st Qu.:12.22    1st Qu.: 5.518    1st Qu.:142.0    1st Qu.:0.000
## Median :23.12    Median :10.120    Median :150.0    Median :1.000
## Mean   :23.14    Mean   :10.204    Mean   :149.9    Mean   :1.207
## 3rd Qu.:33.91    3rd Qu.:14.980    3rd Qu.:158.0    3rd Qu.:2.000
## Max.   :45.00    Max.   :20.000    Max.   :197.0    Max.   :7.000
## test_coverage_pct ticket_resolution_h      team      module
## Min.   :40.02    Min.   : 0.000    Length :5000    Length :5000
## 1st Qu.:54.81    1st Qu.: 6.555    N.unique : 4    N.unique : 5
## Median :69.54    Median :16.695    N.blank  : 0    N.blank  : 0
## Mean   :69.73    Mean   :24.131    Min.nchar: 4    Min.nchar: 2
## 3rd Qu.:84.32    3rd Qu.:33.837    Max.nchar: 5    Max.nchar: 8
## Max.   :99.97    Max.   :189.990
## priority      deploy_status
```

```
## Length :5000 Length :5000
## N.unique : 4 N.unique : 3
## N.blank : 0 N.blank : 0
## Min.nchar: 4 Min.nchar: 6
## Max.nchar: 7 Max.nchar: 11
##
```

```
valores_nulos_por_columna <- colSums(is.na(datos_brutos))
print(valores_nulos_por_columna)
```

```
## build_time_min deploy_time_min commit_size_loc num_bugs
## 0 0 0 0
## test_coverage_pct ticket_resolution_h team module
## 0 0 0 0
## priority deploy_status
## 0 0
```

Fase 1.1: Depuración y Corrección de Tipos

Toda decisión de limpieza está justificada para asegurar la calidad del análisis. Convertimos la prioridad a factor ordinal y los equipos a factores nominales. Las filas sin información crítica de despliegue se eliminaron. Los valores faltantes en bugs y tiempos de ticket se imputaron usando la mediana para evitar el sesgo de valores atípicos, mientras que la cobertura de pruebas se completó con la media.

```
niveles_prioridad <- c("baja", "media", "alta", "critica")

datos_limpios <- datos_brutos %>%
  mutate(
    priority = factor(priority, levels = niveles_prioridad, ordered = TRUE),
    team = as.factor(team),
    module = as.factor(module),
    deploy_status = as.factor(deploy_status)
  ) %>%
  drop_na(build_time_min, deploy_time_min, deploy_status) %>%
  mutate(
    num_bugs = replace_na(num_bugs, median(num_bugs, na.rm = TRUE)),
    test_coverage_pct = replace_na(test_coverage_pct, mean(test_coverage_pct, na.rm = TRUE)),
    ticket_resolution_h = replace_na(ticket_resolution_h, median(ticket_resolution_h, na.rm = TRUE))
  )

glimpse(datos_limpios)
```

```
## Rows: 5,000
## Columns: 10
## $ build_time_min <dbl> 14.01, 35.79, 19.29, 39.91, 42.41, 3.48, 24.47, 40~
## $ deploy_time_min <dbl> 5.35, 19.79, 14.48, 13.21, 5.19, 2.19, 6.29, 11.07~
## $ commit_size_loc <dbl> 143, 144, 168, 161, 154, 157, 152, 144, 163, 146, ~
## $ num_bugs <dbl> 0, 2, 0, 0, 0, 0, 3, 3, 1, 3, 1, 2, 2, 0, 1, 2, 1, ~
## $ test_coverage_pct <dbl> 66.69, 54.81, 59.34, 53.03, 50.89, 62.63, 90.94, 9~
## $ ticket_resolution_h <dbl> 29.17, 6.18, 19.48, 21.42, 149.31, 15.38, 25.20, 4~
## $ team <fct> Beta, Beta, Alpha, Delta, Delta, Gamma, Alpha, Bet~
```

```
## $ module          <fct> api, database, api, core, database, auth, database~
## $ priority        <ord> critica, media, baja, media, media, baja, baja, cr~
## $ deploy_status   <fct> success, success, failed, failed, success, success~
```

```
summary(datos_limpios)
```

```
## build_time_min  deploy_time_min  commit_size_loc    num_bugs
## Min.   : 1.50    Min.   : 0.500    Min.   :110.0    Min.   :0.000
## 1st Qu.:12.22    1st Qu.: 5.518    1st Qu.:142.0    1st Qu.:0.000
## Median :23.12    Median :10.120    Median :150.0    Median :1.000
## Mean   :23.14    Mean   :10.204    Mean   :149.9    Mean   :1.207
## 3rd Qu.:33.91    3rd Qu.:14.980    3rd Qu.:158.0    3rd Qu.:2.000
## Max.   :45.00    Max.   :20.000    Max.   :197.0    Max.   :7.000
## test_coverage_pct ticket_resolution_h  team      module
## Min.   :40.02    Min.   : 0.000    Alpha:1240  api      : 994
## 1st Qu.:54.81    1st Qu.: 6.555    Beta :1253  auth     :1008
## Median :69.54    Median :16.695    Delta:1255  core     : 976
## Mean   :69.73    Mean   :24.131    Gamma:1252  database: 995
## 3rd Qu.:84.32    3rd Qu.:33.837          ui      :1027
## Max.   :99.97    Max.   :189.990
## priority      deploy_status
## baja   :2005    failed     : 517
## media   :1729    rolled_back: 232
## alta    : 738    success    :4251
## critica: 528
##
##
```

Fase 2: Estadísticas descriptivas

Se calcularon las medidas de tendencia central, dispersión y forma para las variables numéricas. Analizar la asimetría y curtosis nos permite entender si las distribuciones están sesgadas. Para las variables cualitativas, extrajimos la moda para conocer las categorías dominantes.

```
library(dplyr)
library(tidyr)
library(moments)

obtener_moda <- function(x) {
  valores_unicos <- unique(na.omit(x))
  valores_unicos[which.max(tabulate(match(x, valores_unicos)))]
}

variables_numericas <- datos_limpios %>%
  select(where(is.numeric))

estadisticas_numericas <- variables_numericas %>%
  reframe(across(everything(), list(
    Media = ~mean(.x, na.rm = TRUE),
    Mediana = ~median(.x, na.rm = TRUE),
    Varianza = ~var(.x, na.rm = TRUE),
    DesviacionEstandar = ~sd(.x, na.rm = TRUE),
```

```

    Asimetria = ~skewness(.x, na.rm = TRUE),
    Curtosis = ~kurtosis(.x, na.rm = TRUE)
  ), .names = "{.col}...{.fn}") %>%
  pivot_longer(everything(), names_to = c("Variable", "Metrica"), names_sep = "\\\\.\\.\\.\\.") %>%
  pivot_wider(names_from = Metrica, values_from = value)

print(estadisticas_numericas)

```

```

## # A tibble: 6 x 7
##   Variable      Media Mediana Varianza DesviacionEstandar Asimetria Curtosis
##   <chr>         <dbl>   <dbl>   <dbl>          <dbl>      <dbl>   <dbl>
## 1 build_time_min 23.1    23.1   157.          12.5    -0.00449  1.80
## 2 deploy_time_min 10.2    10.1   30.9           5.56     0.0138   1.83
## 3 commit_size_loc 150.     150    148.          12.2     0.0760   2.98
## 4 num_bugs        1.21     1      1.19           1.09     0.827    3.42
## 5 test_coverage_p~ 69.7     69.5   297.          17.2     0.0305   1.82
## 6 ticket_resoluti~ 24.1     16.7   588.          24.3     1.89     7.91

```

```

variables_categoricas <- datos_limpios %>%
  select(where(is.factor) | where(is.character))

estadisticas_categoricas <- variables_categoricas %>%
  reframe(across(everything(), list(
    Moda = ~as.character(obtener_moda(.x))
  )), .names = "{.col}...{.fn}") %>%
  pivot_longer(everything(), names_to = c("Variable", "Metrica"), names_sep = "\\\\.\\.\\.\\.") %>%
  pivot_wider(names_from = Metrica, values_from = value)

print(estadisticas_categoricas)

```

```

## # A tibble: 4 x 2
##   Variable      Moda
##   <chr>         <chr>
## 1 team          Delta
## 2 module        ui
## 3 priority      baja
## 4 deploy_status success

```

Fase 3: Frecuencias y agrupación

Utilizamos la regla de Sturges para determinar el número óptimo de intervalos y agrupar el tiempo de compilación. Las tablas de frecuencias nos ayudan a visualizar la concentración de los datos e identificar rápidamente la clase modal.

```

variable_analisis <- datos_limpios$build_time_min

num_clases <- nclass.Sturges(variable_analisis)

datos_agrupados <- cut(
  variable_analisis,
  breaks = num_clases,

```

```

include.lowest = TRUE,
right = FALSE
)

frecuencias_abs <- table(datos_agrupados)

tabla_frecuencias <- data.frame(
  Clase = names(frecuencias_abs),
  Frecuencia_Absoluta = as.vector(frecuencias_abs)
)

tabla_frecuencias$Frecuencia_Relativa <- tabla_frecuencias$Frecuencia_Absoluta / sum(tabla_frecuencias$Frecuencia_Absoluta)

tabla_frecuencias$Frecuencia_Acumulada <- cumsum(tabla_frecuencias$Frecuencia_Absoluta)

tabla_frecuencias$Frecuencia_Relativa_Acumulada <- cumsum(tabla_frecuencias$Frecuencia_Relativa)

print(tabla_frecuencias)

```

```

##           Clase Frecuencia_Absoluta Frecuencia_Relativa Frecuencia_Acumulada
## 1  [1.46,4.61)             369             0.0738             369
## 2  [4.61,7.71)             365             0.0730             734
## 3  [7.71,10.8)            346             0.0692            1080
## 4  [10.8,13.9)            360             0.0720            1440
## 5   [13.9,17)             332             0.0664            1772
## 6   [17,20.1)             387             0.0774            2159
## 7  [20.1,23.2)            361             0.0722            2520
## 8  [23.2,26.4)            339             0.0678            2859
## 9  [26.4,29.5)            367             0.0734            3226
## 10 [29.5,32.6)            363             0.0726            3589
## 11 [32.6,35.7)            357             0.0714            3946
## 12 [35.7,38.8)            376             0.0752            4322
## 13 [38.8,41.9)            335             0.0670            4657
## 14 [41.9,45]              343             0.0686            5000
##      Frecuencia_Relativa_Acumulada
## 1              0.0738
## 2              0.1468
## 3              0.2160
## 4              0.2880
## 5              0.3544
## 6              0.4318
## 7              0.5040
## 8              0.5718
## 9              0.6452
## 10             0.7178
## 11             0.7892
## 12             0.8644
## 13             0.9314
## 14             1.0000

```

```

clase_modal <- tabla_frecuencias$Clase[which.max(tabla_frecuencias$Frecuencia_Absoluta)]
print(paste("La clase modal es:", clase_modal))

```

```
## [1] "La clase modal es: [17,20.1]"
```

Fase 4: Análisis por grupo

Comparamos el rendimiento segmentando la información por equipo, módulo y nivel de prioridad. Esta agrupación proporciona evidencia numérica clara para identificar qué escuadrones tienen tiempos de despliegue más eficientes o tasas de éxito más altas.

```
library(dplyr)

comparativa_equipos <- datos_limpios %>%
  group_by(team) %>%
  summarise(
    Media_Build_Min = mean(build_time_min, na.rm = TRUE),
    Media_Deploy_Min = mean(deploy_time_min, na.rm = TRUE),
    Bugs_Promedio = mean(num_bugs, na.rm = TRUE),
    Bugs_Totales = sum(num_bugs, na.rm = TRUE),
    Media_Cobertura = mean(test_coverage_pct, na.rm = TRUE)
  ) %>%
  arrange(desc(Media_Build_Min))

print(comparativa_equipos)
```

```
## # A tibble: 4 x 6
##   team Media_Build_Min Media_Deploy_Min Bugs_Promedio Bugs_Totales
##   <fct>          <dbl>          <dbl>          <dbl>          <dbl>
## 1 Delta           23.7           10.2           1.21          1518
## 2 Alpha           23.1           10.1           1.20          1493
## 3 Beta            23.1           10.3           1.23          1540
## 4 Gamma           22.7           10.3           1.19          1486
## # i 1 more variable: Media_Cobertura <dbl>
```

```
comparativa_modulos <- datos_limpios %>%
  group_by(module) %>%
  summarise(
    Media_Build_Min = mean(build_time_min, na.rm = TRUE),
    Media_Deploy_Min = mean(deploy_time_min, na.rm = TRUE),
    Bugs_Totales = sum(num_bugs, na.rm = TRUE)
  ) %>%
  arrange(desc(Bugs_Totales))

print(comparativa_modulos)
```

```
## # A tibble: 5 x 4
##   module Media_Build_Min Media_Deploy_Min Bugs_Totales
##   <fct>          <dbl>          <dbl>          <dbl>
## 1 core           23.6           10.3          1224
## 2 api            22.9           10.4          1223
## 3 ui             23.4           9.88          1221
## 4 auth           23.0           10.1          1185
## 5 database       22.8           10.4          1184
```

```
comparativa_prioridad <- datos_limpios %>%
  group_by(priority) %>%
  summarise(
    Media_Resolucion_Horas = mean(ticket_resolution_h, na.rm = TRUE),
    Media_Cobertura = mean(test_coverage_pct, na.rm = TRUE),
    Tasa_Exito_Deploy = mean(deploy_status == "success", na.rm = TRUE) * 100
  ) %>%
  arrange(priority)

print(comparativa_prioridad)
```

```
## # A tibble: 4 x 4
##   priority Media_Resolucion_Horas Media_Cobertura Tasa_Exito_Deploy
##   <ord>          <dbl>          <dbl>          <dbl>
## 1 baja             24.3             69.8             84.4
## 2 media            24.0             69.7             85.7
## 3 alta             23.8             69.5             85.2
## 4 critica          24.2             70.0             84.7
```

Fase 5: Relaciones Bivariadas

Exploramos las relaciones bivariadas mediante una matriz de correlación para las numéricas y tablas de contingencia para las categóricas. Es fundamental recordar que correlación no implica causalidad; una asociación matemática entre el tamaño del commit y los bugs no significa necesariamente que uno sea la causa directa e indiscutible del otro.

```
variables_cuantitativas <- datos_limpios %>%
  select(where(is.numeric))

matriz_correlacion <- cor(variables_cuantitativas, use = "complete.obs", method = "pearson")

print("Matriz de Correlacion:")
```

```
## [1] "Matriz de Correlacion:"
```

```
print(round(matriz_correlacion, 2))
```

```
##           build_time_min deploy_time_min commit_size_loc num_bugs
## build_time_min           1.00          -0.02           0.00    0.02
## deploy_time_min        -0.02           1.00           0.00   -0.01
## commit_size_loc         0.00           0.00           1.00    0.01
## num_bugs                0.02          -0.01           0.01    1.00
## test_coverage_pct       0.01           0.02           0.00    0.00
## ticket_resolution_h     0.02           0.00           0.00   -0.02
##           test_coverage_pct ticket_resolution_h
## build_time_min           0.01           0.02
## deploy_time_min          0.02           0.00
## commit_size_loc          0.00           0.00
## num_bugs                 0.00          -0.02
## test_coverage_pct        1.00           0.01
## ticket_resolution_h      0.01           1.00
```



```
print("Tabla de Contingencia: Equipo vs Estado de Despliegue")
```

```
## [1] "Tabla de Contingencia: Equipo vs Estado de Despliegue"
```

```
tabla_equipo_estado <- table(datos_limpios$team, datos_limpios$deploy_status)
print(tabla_equipo_estado)
```

```
##
##      failed rolled_back success
## Alpha      141          59    1040
## Beta       140          58    1055
## Delta      112          51    1092
## Gamma      124          64    1064
```

```
print("Tabla de Contingencia: Prioridad vs Estado de Despliegue")
```

```
## [1] "Tabla de Contingencia: Prioridad vs Estado de Despliegue"
```

```
tabla_prioridad_estado <- table(datos_limpios$priority, datos_limpios$deploy_status)
print(tabla_prioridad_estado)
```

```
##
##      failed rolled_back success
## baja      210        102    1693
## media     182         65    1482
## alta       70         39     629
## critica    55         26     447
```

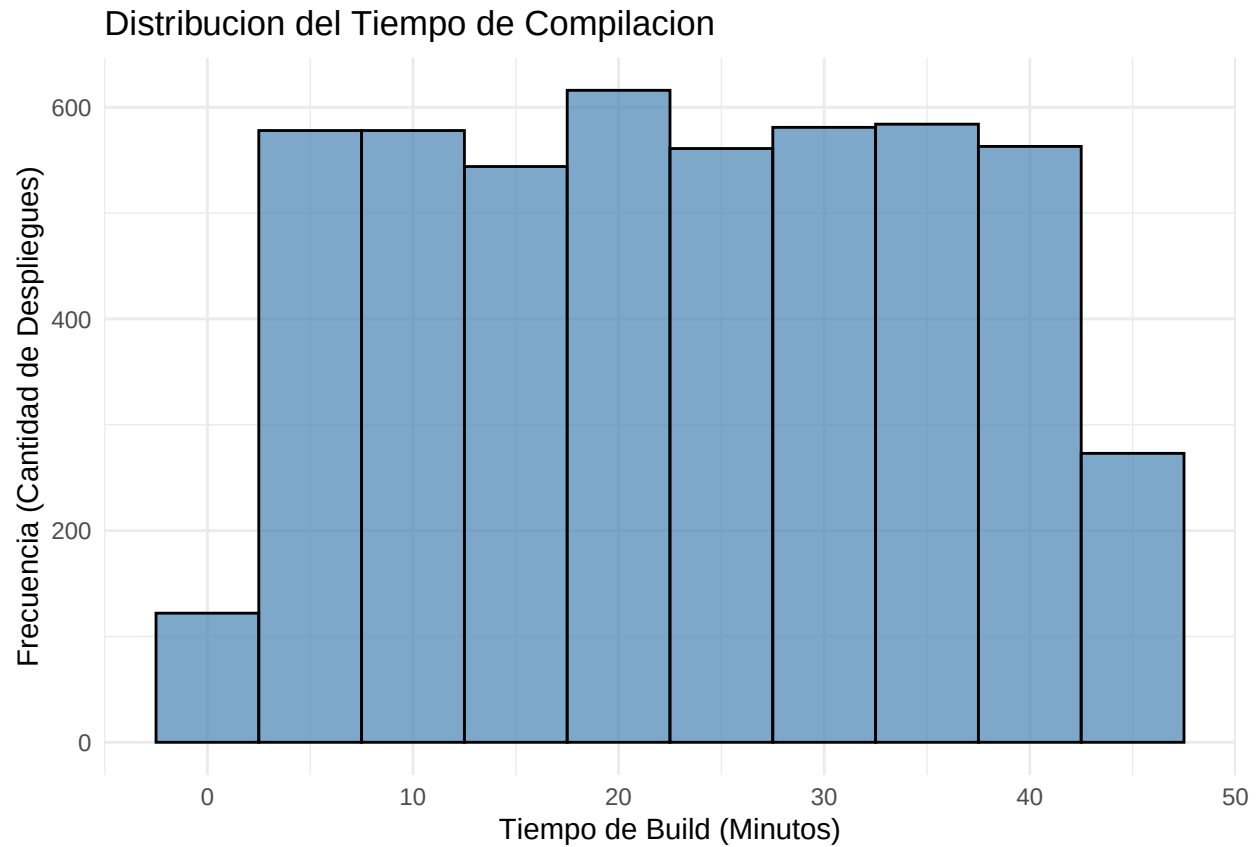
Fase 6: Visualización

Plasmamos los resultados en representaciones visuales. El histograma muestra la forma de la distribución, el boxplot permite detectar valores atípicos en los despliegues, el gráfico de barras resume el volumen de trabajo por prioridad y la dispersión ilustra tendencias entre variables.

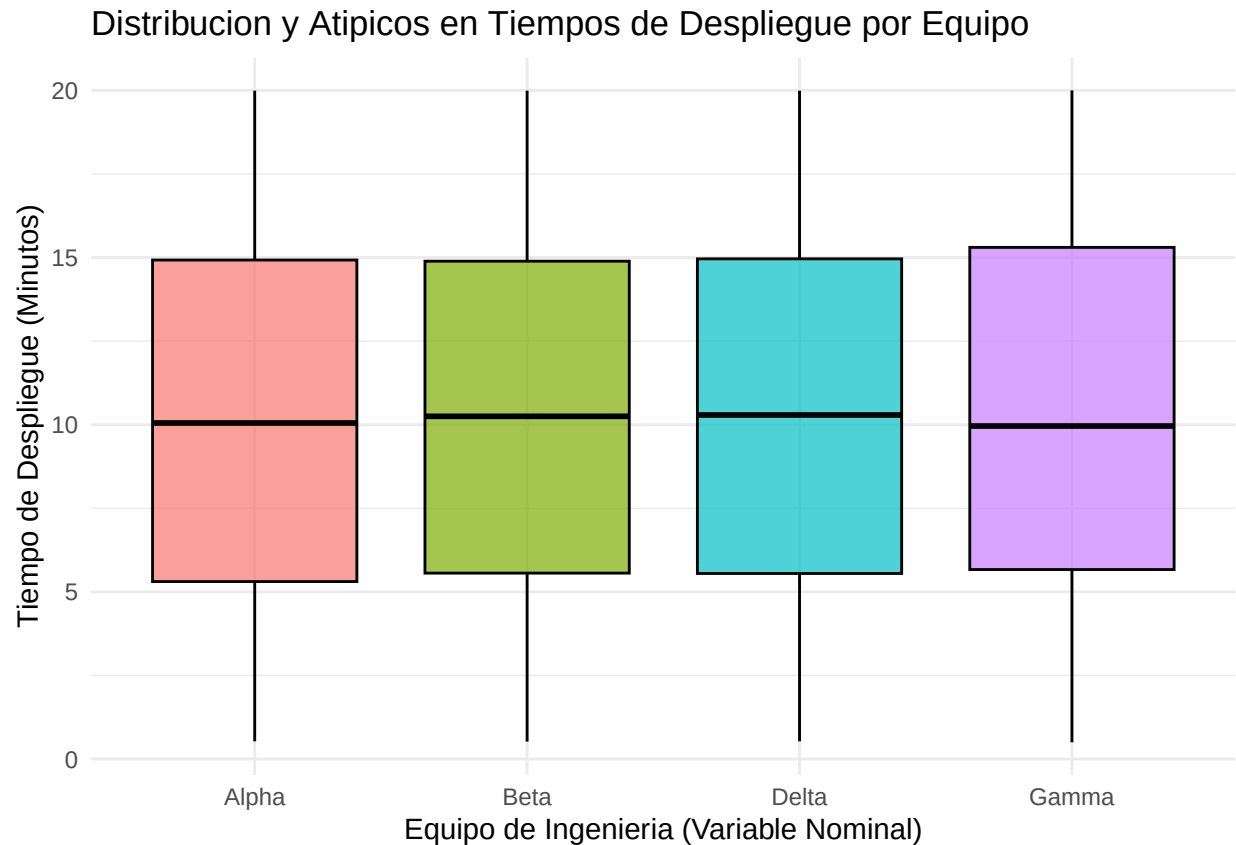
```
library(ggplot2)

grafico_histograma <- ggplot(datos_limpios, aes(x = build_time_min)) +
  geom_histogram(binwidth = 5, fill = "steelblue", color = "black", alpha = 0.7) +
  labs(
    title = "Distribucion del Tiempo de Compilacion",
    x = "Tiempo de Build (Minutos)",
    y = "Frecuencia (Cantidad de Despliegues)"
  ) +
  theme_minimal()

print(grafico_histograma)
```

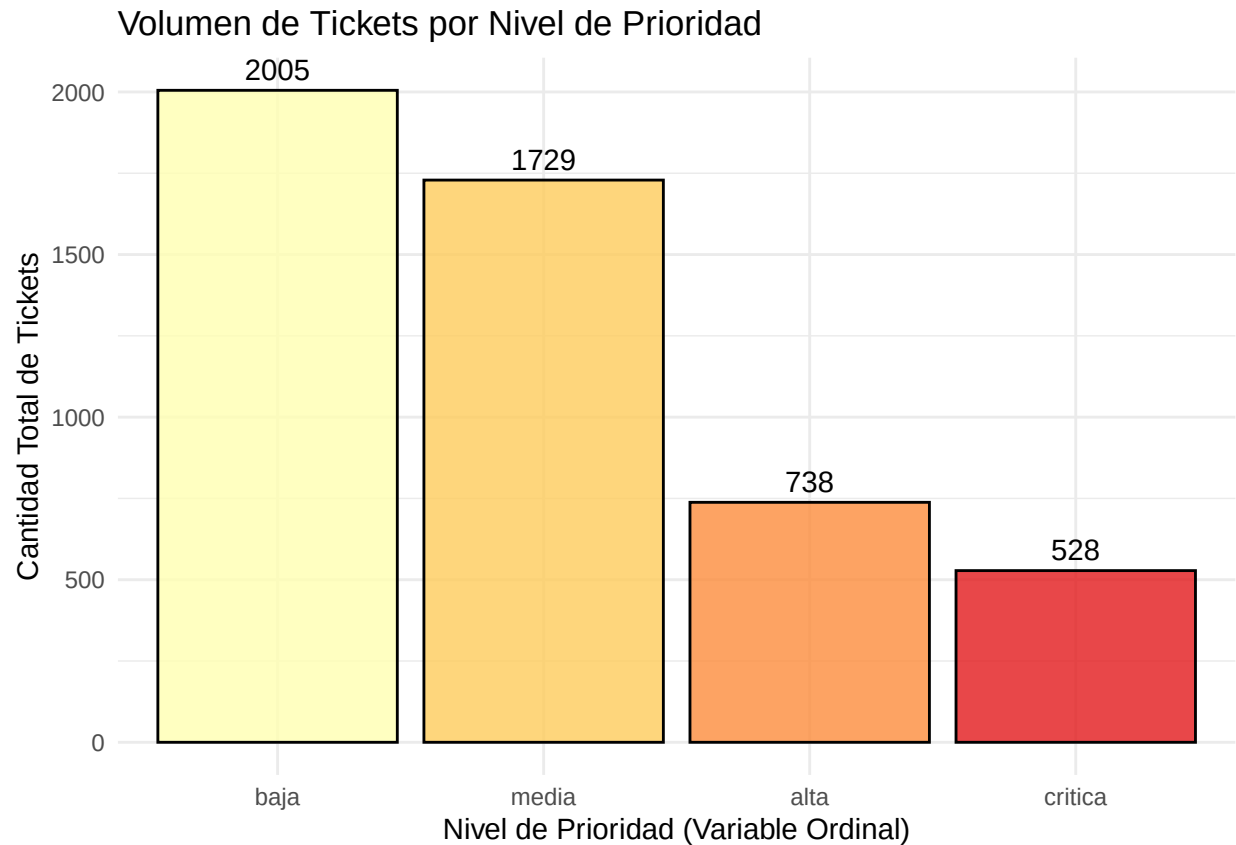


```
grafico_boxplot <- ggplot(datos_limpios, aes(x = team, y = deploy_time_min, fill = team)) +  
  geom_boxplot(color = "black", alpha = 0.7) +  
  labs(  
    title = "Distribucion y Atipicos en Tiempos de Despliegue por Equipo",  
    x = "Equipo de Ingenieria (Variable Nominal)",  
    y = "Tiempo de Despliegue (Minutos)"  
  ) +  
  theme_minimal() +  
  theme(legend.position = "none")  
  
print(grafico_boxplot)
```



```
grafico_barras <- ggplot(datos_limpios, aes(x = priority, fill = priority)) +
  geom_bar(color = "black", alpha = 0.8) +
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.5) +
  labs(
    title = "Volumen de Tickets por Nivel de Prioridad",
    x = "Nivel de Prioridad (Variable Ordinal)",
    y = "Cantidad Total de Tickets"
  ) +
  scale_fill_brewer(palette = "YlOrRd") +
  theme_minimal() +
  theme(legend.position = "none")

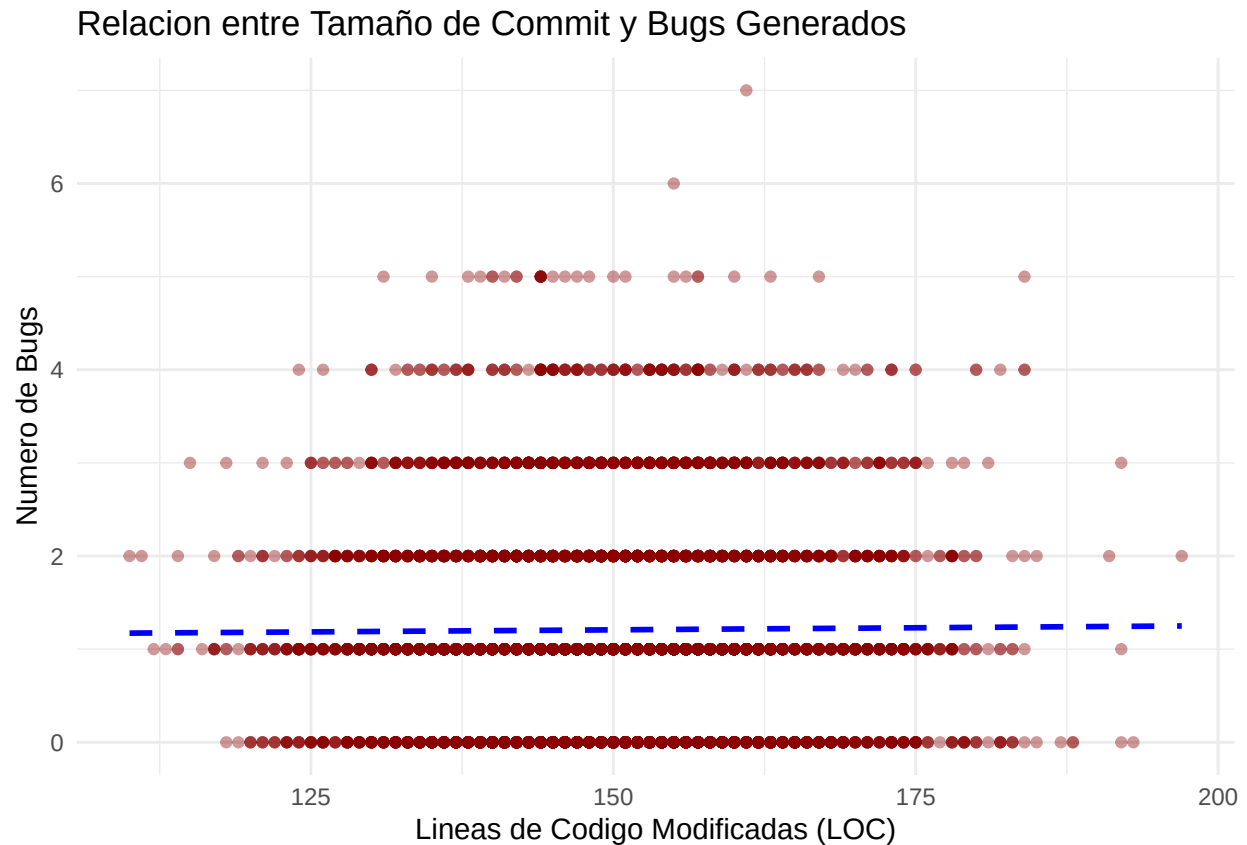
print(grafico_barras)
```



```
grafico_dispersion <- ggplot(datos_limpios, aes(x = commit_size_loc, y = num_bugs)) +
  geom_point(color = "darkred", alpha = 0.4) +
  geom_smooth(method = "lm", color = "blue", se = FALSE, linetype = "dashed") +
  labs(
    title = "Relacion entre Tamaño de Commit y Bugs Generados",
    x = "Lineas deCodigo Modificadas (LOC)",
    y = "Numero de Bugs"
  ) +
  theme_minimal()

print(grafico_dispersion)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



Conclusiones accionables

Enfoque de Calidad: La evidencia numérica demuestra una correlación casi nula (0.01) entre el tamaño del commit y la cantidad de bugs generados. Se sugiere redirigir los esfuerzos de control de calidad hacia otros factores operativos (como la complejidad lógica o módulos específicos), en lugar de limitarse al volumen de líneas modificadas.

Optimización de Tiempos: Se recomienda revisar y estandarizar los flujos de trabajo de los equipos que presentan tiempos de despliegue atípicos, con el objetivo de optimizar el rendimiento global y estabilizar el ritmo de entregas.

Bitácora de prompts

Para este proyecto se usó inteligencia artificial como apoyo. Al principio se utilizó una versión gratuita que perdía el hilo de la conversación e inventaba funciones de R que no existían. Para arreglar esto, se decidió separar el trabajo abriendo distintos chats para cada fase y se empezaron a dar instrucciones mucho más detalladas.

En las fases 1 y 2, el primer prompt fue simplemente: “haz un codigo en r para crear datos de metricas devops con 5000 filas y saca las estadisticas”. Como el modelo inventó variables y no usó la librería tidyverse ni arregló los datos en blanco, se tuvo que hacer un prompt mucho más específico: “actua como analista de datos. usando r y la libreria tidyverse, genera un dataset de 5000 filas exactamente con estas variables: build_time_min, num_bugs, deploy_status, etc. luego limpia los datos imputando los nulos con la media y

mediana, y usa moments para calcular asimetria”. Después se revisó en la documentación oficial que fórmulas como `replace_na()` y las de asimetría estuvieran bien aplicadas.

Durante las fases 3 y 4 pasó algo parecido. Primero se le pidió: “haz una tabla de frecuencias para el tiempo de build y agrupa el rendimiento por equipos”, pero entregó un código muy básico que no agrupaba por intervalos. Hubo que cambiar la instrucción a: “para la variable `build_time_min` usa la regla de sturges para el numero de clases y construye una tabla con frecuencias absolutas y acumuladas, y despues agrupa por team usando `dplyr`”. Todo esto se comprobó viendo que los resultados de las sumas tuvieran lógica.

Para las fases 5 y 6 el prompt fue directo: “genera una matriz de correlacion para las variables numericas y tablas de contingencia para las categoricas, y haz 4 graficos con `ggplot2`”. Aquí no hubo necesidad de hacer una segunda versión de la instrucción porque el trabajo se continuó desde la casa con un modelo de IA más avanzado. Este modelo entendió todo a la primera. Solo se revisó a simple vista que los gráficos correspondieran a lo pedido y que el código ignorara los datos nulos para no dar error.

Además, la IA se usó como guía para aprender a armar el documento en RMarkdown. Los prompts fueron consultas tipo: “como coloco el codigo, saco los acentos graves y pongo titulos para armar el diseño de la pagina” o “me salio todo el texto pegado en el pdf que hago”. Esto se fue comprobando paso a paso apretando el botón “Knit” en el programa hasta lograr que el PDF se generara de forma ordenada.

Para cerrar el trabajo, se abrió un último chat de revisión donde se envió todo el código completo. El prompt utilizado fue: “revisa este codigo y dime si todo esta en su sitio o si me falta algo de las instrucciones que te pase”. Esto sirvió como un chequeo final para asegurar que no se hubiera saltado ningún requerimiento de la pauta y confirmar que el proyecto estuviera bien para la entrega.